

STM32 嵌入式开发环境配置

适用场景：已完成 WSL2 开发环境搭建，需要在此基础上配置 STM32 嵌入式开发工具链。目标是搭建一套 **AI 原生友好** 的开发环境——项目结构全文本化、构建全命令行化，AI 可以直接读写源码和构建配置、执行编译、解析报错并修复。

前置条件：请先完成 [02-wsl2-setup.md](#) 中的全部配置。

这份文档做什么：

- 1. 在 WSL 里安装 STM32 交叉编译工具链（ARM GCC、CMake、Ninja、OpenOCD 等）
- 2. 在 Windows 侧安装 STM32CubeMX（项目骨架生成器）和 usbipd-win（USB 透传）
- 3. 跑通"CubeMX 生成项目 → CMake 配置 → Ninja 编译出固件"的完整流程
- 4. 跑通"USB 透传 → OpenOCD 烧录到板子"的硬件链路

为什么不用 Keil：

Keil 的项目配置（**.uvprojx**）、编译器（armcc/armclang）、构建过程全部封装在 GUI 里，AI 无法读取、无法修改、无法调用。本文档搭建的工具链，每一个环节都是**命令行 + 文本配置**，AI 可以完全参与"改代码 → 编译 → 看报错 → 修复"的开发循环。

核心工具链选型：

环节	选择	替代了 Keil 的什么
SDK / HAL	STM32CubeMX + HAL 库	Keil 的 Pack Manager
构建系统	CMake	Keil 的 .uvprojx 工程文件
编译器	arm-none-eabi-gcc	Keil 的 armcc / armclang
构建执行器	Ninja（Make 备用）	Keil 的"Build"按钮
烧录	OpenOCD（CubeProgrammer 备用）	Keil 的"Download"按钮
USB 透传	usbipd-win	不需要（Keil 在 Windows 原生运行）

预估耗时和空间：

阶段	耗时	磁盘占用
一（WSL 侧工具链安装）	5-10 分钟	~500 MB
二（Windows 侧工具安装）	10-20 分钟	~2 GB
三（验证开发工作流）	10-15 分钟	取决于项目
四（配通烧录链路）	5-10 分钟	—

风险等级说明（同 02 文档）：

等级	含义
----	----

等级	含义
●	无风险：纯库/工具，不改系统配置，卸载干净
○	低风险：会拉少量依赖，但都是成熟稳定包

关于代码块的执行方式（同 02 文档）：

- 📄 整块复制粘贴执行
- ⌘ 逐条复制粘贴执行（前一条会改变环境，后续依赖变更）
- 📄 粘贴到编辑器中的配置内容（不是终端命令）

一、WSL 侧工具安装

以下所有命令在 **WSL 终端** 中执行。

1.1 ARM GCC 交叉编译器 ● 无风险

为什么要装：这是把 C 代码编译成 STM32 能运行的机器码的核心工具。

名字拆解：**arm**（ARM 架构）+ **none**（裸机，无操作系统）+ **eabi**（嵌入式二进制接口标准）+ **gcc**（GCC 编译器）。

📄 执行：

```
sudo apt install -y gcc-arm-none-eabi binutils-arm-none-eabi libnewlib-arm-none-eabi
```

各包说明：

包	说明
<code>gcc-arm-none-eabi</code>	交叉编译器本体，把 C/C++ 编译成 ARM Cortex-M 机器码
<code>binutils-arm-none-eabi</code>	二进制工具集： <code>objcopy</code> （.elf → .bin/.hex 格式转换）、 <code>objdump</code> （反汇编）、 <code>size</code> （查看各段大小）、 <code>nm</code> （符号表）
<code>libnewlib-arm-none-eabi</code>	嵌入式 C 标准库（ <code>printf</code> 、 <code>malloc</code> 等函数的轻量实现）

📄 验证：

```
arm-none-eabi-gcc --version
```

关于版本：Ubuntu 24.04 apt 源中的版本足够日常 STM32 开发。如果未来需要最新版本（比如支持最新芯片或新的编译器优化），可以从 [ARM 官方](#) 下载，解压到 `/opt/` 后加入 PATH 即可替换。

1.2 CMake + Ninja 构建工具 ● 无风险

为什么要装：CMake 是构建系统——用 `CMakeLists.txt` 文本文件描述“哪些文件要编译、用什么参数、怎么链接”。Ninja 是构建执行器——读取 CMake 生成的指令，实际执行编译。CubeMX 可以直接导出 CMake 项目，两者配合开箱即用。

📄 执行：

```
sudo apt install -y cmake ninja-build
```

📄 验证：

```
cmake --version && ninja --version
```

说明：`make` 已在 02 文档的 `build-essential` 中安装，可作为 Ninja 的备用执行器。默认使用 Ninja，因为它的输出更干净（单行进度条 + 错误信息），AI 解析编译结果更容易。

1.3 GDB 多架构调试器 ● 无风险

为什么要装：GDB 配合 OpenOCD 可以在线调试 STM32——打断点、查变量、单步执行。`gdb-multiarch` 支持所有 CPU 架构，比单一的 `gdb-arm-none-eabi` 更通用（Ubuntu 24.04 apt 源中可能没有后者）。

📄 执行：

```
sudo apt install -y gdb-multiarch
```

📄 验证：

```
gdb-multiarch --version
```

使用方式：调试时执行 `gdb-multiarch build/firmware.elf`，然后在 GDB 里连接 OpenOCD 的 GDB 服务端口（默认 3333）。具体调试流程在第四节说明。

1.4 OpenOCD 烧录/调试服务器 ● 无风险

为什么要装：OpenOCD 通过调试器硬件（ST-Link）与 STM32 芯片通信，完成烧录和调试。它是“翻译官”——把你的命令翻译成 ST-Link 能理解的 SWD/JTAG 协议。

📄 执行：

```
sudo apt install -y openocd
```

📄 验证:

```
openocd --version
```

OpenOCD 安装后自带几百个配置文件模板，覆盖了几乎所有常见调试器和芯片型号：

路径	用途	常用文件举例
/usr/share/openocd/scripts/interface/	调试器配置	stlink.cfg (ST-Link V2/V3)
/usr/share/openocd/scripts/target/	芯片配置	stm32f1x.cfg、stm32f4x.cfg、stm32h7x.cfg

烧录时只需指定两个配置文件——调试器 + 芯片型号。一行命令完成烧录：

```
openocd -f interface/stlink.cfg -f target/stm32f4x.cfg \
-c "program build/firmware.elf verify reset exit"
```

1.5 串口通信工具 🌀 无风险

为什么要装：嵌入式开发中 UART 串口是最常用的调试输出方式——在代码里用 `printf` 把信息打印到串口，在电脑上用串口工具查看。

📄 执行:

```
sudo apt install -y minicom picocom
```

工具	特点
minicom	功能全面，有配置菜单，类似 Windows 下的串口调试助手
picocom	极简轻量，适合快速查看串口输出。用法： <code>picocom -b 115200 /dev/ttyACM0</code>

📄 验证:

```
minicom --version && picocom --help 2>&1 | head -1
```

串口使用前提：WSL2 下使用串口同样需要 `usbipd-win` 透传 USB 转串口设备（如 CH340、CP2102）。透传后设备出现在 `/dev/ttyACM*` 或 `/dev/ttyUSB*`。

1.6 安装总览

以上所有工具也可以一条命令装完。如果你跳过了前面逐个安装的步骤，📄 执行这一条即可：

```
sudo apt install -y \
    gcc-arm-none-eabi binutils-arm-none-eabi libnewlib-arm-none-eabi \
    cmake ninja-build \
    gdb-multiarch \
    openocd \
    minicom picocom
```

二、Windows 侧工具安装

以下操作在 **Windows** 上进行。

2.1 STM32CubeMX (项目骨架生成器)

为什么要装：CubeMX 是 ST 官方的图形化工具，用来**生成 STM32 项目骨架**——选芯片型号、配引脚、配时钟和外设，然后一键生成包含启动文件、链接脚本、HAL 库源码、**CMakeLists.txt** 的完整项目目录。

和 AI 的关系：CubeMX 只在**创建新项目或修改外设配置**时用（低频操作），生成完之后的日常开发（改代码、改构建配置、编译、修 bug）全在文本文件里进行，AI 完全可以接管。

安装步骤：

1. 访问 [STM32CubeMX 下载页](#)
2. 需要注册免费的 myST 账号（邮箱即可）
3. 下载 Windows 版安装包（.exe），运行安装，一路默认即可
4. 安装完成后从开始菜单打开 STM32CubeMX 确认能正常启动

版本要求：6.14.0 或更高。CMake 项目导出是 6.14.0 新增的功能，旧版本不支持。当前最新版可在[下载页](#)查看。

关于 STM32CubeCLT：ST 还提供了 [STM32CubeCLT](#) (Command Line Tools)，捆绑了 ARM GCC、CMake、Ninja、STM32CubeProgrammer CLI 等工具。如果你希望 Windows 侧也有一套完整工具链（比如在 Windows 原生环境下编译），可以装这个。但对于本文档的 WSL 开发路线，**WSL 侧已经通过 apt 装好了这些工具，CubeCLT 不是必须的。**

2.2 usbipd-win (USB 透传)

为什么要装：WSL2 本质是虚拟机，默认看不到插在电脑上的 USB 设备。ST-Link 调试器和 USB 转串口（CH340 等）都是 USB 设备，要让 WSL 里的 OpenOCD / picocom 能访问它们，需要把 USB 设备“透传”进 WSL。usbipd-win 是微软推荐的官方方案。

安装：

打开 **PowerShell (管理员)**，📄 执行：

```
winget install --interactive --exact dorssel.usbipd-win
```

安装完成后**重启电脑**（usbipd 需要安装一个系统服务）。

📖 验证（重启后打开 PowerShell）：

```
usbipd --version
```

说明：新版 usbipd-win (4.0+) **WSL 侧不需要安装任何东西**。以前的版本需要在 WSL 里装 usbip 客户端和内核驱动，现在已经不需要了，WSL 内核自带支持。

参考：[微软官方文档](#) | [usbipd-win GitHub](#)

三、开发工作流：从新建项目到编译成功

本节跑通一个最小完整流程：CubeMX 生成项目 → 复制到 WSL → CMake 配置 → Ninja 编译 → 得到 `.elf` 固件文件。

3.1 CubeMX 生成 CMake 项目

在 **Windows** 上打开 STM32CubeMX，按以下步骤操作：

① 新建项目

- 点击 **ACCESS TO MCU SELECTOR** 或 **ACCESS TO BOARD SELECTOR**
- 搜索并选择你的芯片型号（如 **STM32F103C8**）或开发板（如 **NUCLEO-F411RE**）
- 点击 **Start Project**

② 配置外设（按需）

- 左侧树形菜单选择需要的外设（GPIO、UART、SPI、TIM 等）
- 配置引脚分配、时钟树等
- 初次使用可以只配一个 GPIO 输出（用来做最简单的 LED 闪烁验证）

③ 项目设置（关键步骤）

- 点击顶部 **Project Manager** 标签页
- **Project Name**：填项目名（如 **blink**）
- **Project Location**：选一个 Windows 路径（如 **C:\STM32Projects**）
- **Toolchain / IDE**：**选择 CMake**（这是关键——必须选 CMake，不要选 MDK-ARM 或 STM32CubeIDE）

④ 生成代码

- 点击右上角 **GENERATE CODE**
- 生成完成后点 **Open Folder** 查看项目目录

3.2 将项目复制到 WSL 文件系统

为什么要复制：WSL 访问 Windows 文件系统（`/mnt/c/`）的 I/O 性能很差，编译速度会慢好几倍。把项目放到 WSL 自己的文件系统（`~/`）下，编译速度正常。

假设 CubeMX 生成的项目在 `C:\STM32Projects\blink`， 在 WSL 终端执行：

```
cp -r /mnt/c/STM32Projects/blink ~/projects/blink
cd ~/projects/blink
```

替代方案：也可以让 CubeMX 直接生成到 WSL 文件系统。在 Project Location 里填 `\\wsl$\\Ubuntu\\home\\你的用户名\\projects\\blink`。但部分 CubeMX 版本对 UNC 路径支持不好，复制更稳妥。

注意同步：如果后续在 CubeMX 中修改了外设配置并重新生成代码，生成的还是 Windows 路径下那份。**你需要重新复制到 WSL**，否则编译的是旧配置。建议每次重新生成后执行：

```
rm -rf ~/projects/blink && cp -r /mnt/c/STM32Projects/blink ~/projects/blink
```

3.3 修复编译器路径（首次必做）

为什么要这一步：CubeMX 生成的工具链文件里，编译器路径很可能是硬编码的绝对路径（指向 CubeCLT 的安装位置），而不是直接写 `arm-none-eabi-gcc`。如果你没装 CubeCLT，编译会直接报“找不到编译器”。这个坑几乎 100% 会遇到。

 先检查工具链文件中的编译器路径：

```
grep -r "arm-none-eabi-gcc" cmake/ CMakeLists.txt 2>/dev/null | head -5
```

如果看到类似这样的**绝对路径**：

```
/opt/st/stm32cubeclt/GNU-tools-for-STM32/bin/arm-none-eabi-gcc
```

需要改成不带路径的 `arm-none-eabi-gcc`（让系统从 PATH 中查找 apt 安装的版本）。具体改哪个文件取决于 CubeMX 版本——可能在 `cmake/gcc-arm-none-eabi.cmake`、`CMakeLists.txt`、或 `CMakePresets.json` 中。把所有 `/opt/st/stm32cubeclt/.../bin/` 前缀删掉即可。

验证编译器可被找到：

```
which arm-none-eabi-gcc
# 应输出 /usr/bin/arm-none-eabi-gcc
```

3.4 CMake 配置与编译

进入项目目录后， **逐条执行：**

```
# 创建构建目录
cmake -B build -G Ninja
```

```
# 编译
cmake --build build
```

说明：

- **-B build**：在 **build/** 子目录下生成构建文件，保持源码目录干净
- **-G Ninja**：使用 Ninja 作为执行器（换成 **-G "Unix Makefiles"** 就用 Make）
- CubeMX 生成的 **CMakeLists.txt** 里已经配好了交叉编译工具链（指向 **arm-none-eabi-gcc**），不需要手动指定

编译成功后，固件文件在：

文件	用途
build/项目名.elf	包含调试信息的完整固件， GDB 调试 和 OpenOCD 烧录 都用这个
build/项目名.bin	纯二进制固件，不含调试信息
build/项目名.hex	Intel HEX 格式，部分烧录工具需要

📁 验证编译产物：

```
ls -lh build/*.elf build/*.bin build/*.hex 2>/dev/null
arm-none-eabi-size build/*.elf
```

arm-none-eabi-size 会显示固件各段的大小（text = 代码段, data = 已初始化数据, bss = 未初始化数据），用来评估 Flash 和 RAM 占用。

3.5 日常开发循环

项目跑通后，日常开发循环是这样的：



人的工作

烧录到板子 → 观察硬件现象 → 告诉 AI 哪里不对

AI 能做的事：

- 读写所有源码文件（.c、.h）
- 读写构建配置（CMakeLists.txt）
- 执行编译（cmake --build build）
- 解析编译错误并自动修复
- 查看固件大小（arm-none-eabi-size）
- 读写 OpenOCD 配置文件

需要人做的事：

- 用 CubeMX 初始化项目 / 修改外设配置（低频，改完重新生成即可）
- 烧录固件到板子（一行命令）
- 观察硬件现象（LED、电机、串口输出等）

重新生成代码：如果在 CubeMX 中修改了外设配置并重新生成，CubeMX 只会覆盖它管理的文件（HAL 配置、启动文件等），**不会覆盖你在 /* USER CODE BEGIN */ 和 /* USER CODE END */ 之间写的代码。**这是 CubeMX 的核心设计——用注释标记保护用户代码。

3.6 一键构建+烧录脚本（可选）

编译成功后烧录虽然只是一行命令，但每次手敲还是麻烦。可以在项目根目录放一个脚本，把构建和烧录串起来。

📁 在项目根目录创建 `flash.sh`，内容如下（根据你的芯片型号修改 target 配置文件）：

```
#!/bin/bash
set -e

# ===== 按你的实际情况修改这两行 =====
OPENOCD_INTERFACE="interface/stlink.cfg"
OPENOCD_TARGET="target/stm32f1x.cfg"
# =====

echo ">>> 编译中..."
cmake --build build

ELF_FILE=$(find build -maxdepth 1 -name "*.elf" | head -1)
if [ -z "$ELF_FILE" ]; then
    echo "!!! 未找到 .elf 文件，编译可能失败"
    exit 1
fi

echo ">>> 固件大小: "
arm-none-eabi-size "$ELF_FILE"
```

```
echo ""
echo ">>> 烧录中..."
openocd -f "$OPENOCD_INTERFACE" -f "$OPENOCD_TARGET" \
  -c "program $ELF_FILE verify reset exit"

echo ">>> 完成! 芯片已复位运行。"
```

📄 赋予执行权限:

```
chmod +x flash.sh
```

使用方式:

命令	用途
<code>cmake --build build</code>	只编译, 不烧录 (AI 日常使用)
<code>./flash.sh</code>	编译 + 烧录一步完成 (人手动使用)

说明: `set -e` 表示任何一步失败就立即停止。如果编译报错, 不会继续烧录。

四、烧录

优先级说明: 烧录链路不阻塞 AI 开发。如果你暂时不需要烧录到实际硬件 (比如先搭环境、先写代码), 可以跳过本节, 以后再回来配。

4.1 USB 透传 (将 ST-Link 映射进 WSL)

插上 ST-Link 调试器后, 在 **Windows PowerShell (管理员)** 中操作:

① 查看已连接的 USB 设备

📄 执行:

```
usbipd list
```

输出示例:

BUSID	VID:PID	DEVICE	STATE
1-2	0483:3748	STM32 STLink	Not shared

记住 ST-Link 的 **BUSID** (上例中是 **1-2**) 。

② 首次绑定 (每个设备只需一次)

📄 执行（替换 1-2 为你的实际 BUSID）：

```
usbipd bind --busid 1-2
```

③ 映射到 WSL

📄 执行：

```
usbipd attach --wsl --busid 1-2
```

④ 在 WSL 中验证

📄 切换到 WSL 终端，执行：

```
lsusb | grep -i st-link
```

看到 STMicroelectronics ST-LINK 字样即透传成功。

日常使用：

- 每次拔插 ST-Link 后需要重新执行 `usbipd attach --wsl --busid <ID>`
- 可以加 `--auto-attach` 参数实现自动重新映射：`usbipd bind --busid 1-2 --auto-attach`（需要 `usbipd-win 4.1+`）
- 用完后可以 `usbipd detach --busid 1-2`，或者直接拔掉 USB 线

lsusb 未找到：如果 `lsusb` 命令不存在，📄 安装：`sudo apt install -y usbutils`

4.2 OpenOCD 烧录

USB 透传成功后，在 **WSL 终端** 中执行烧录。

📄 执行（根据你的芯片型号替换 target 配置文件）：

```
openocd -f interface/stlink.cfg -f target/stm32f1x.cfg \  
-c "program build/firmware.elf verify reset exit"
```

命令含义：

部分	说明
-f interface/stlink.cfg	使用 ST-Link 调试器
-f target/stm32f1x.cfg	目标芯片是 STM32F1 系列（换成你的芯片系列）
program build/firmware.elf	烧录这个固件文件

部分	说明
verify	烧录后校验是否正确
reset	烧录完成后复位芯片，立即开始运行
exit	完成后退出 OpenOCD

常见芯片系列对应的 target 配置文件：

芯片系列	配置文件	常见型号
STM32F1	target/stm32f1x.cfg	F103C8（Blue Pill）、F103RCT6
STM32F4	target/stm32f4x.cfg	F401、F407、F411
STM32H7	target/stm32h7x.cfg	H743、H750
STM32G4	target/stm32g4x.cfg	G431、G474
STM32L4	target/stm32l4x.cfg	L432、L476

烧录成功输出：看到 `verified` 和 `shutdown command invoked` 即表示烧录成功，芯片已复位开始运行新固件。

4.3 GDB 在线调试（按需）

如果需要打断点、单步执行、查看变量，按以下步骤操作。

终端 1 — 启动 OpenOCD 服务（保持运行）：

```
openocd -f interface/stlink.cfg -f target/stm32f1x.cfg
```

终端 2 — 启动 GDB 并连接：

```
gdb-multiarch build/firmware.elf
```

在 GDB 提示符中输入：

```
target remote :3333
monitor reset halt
break main
continue
```

命令	说明
target remote :3333	连接 OpenOCD 的 GDB 服务端

命令	说明
<code>monitor reset halt</code>	复位芯片并暂停
<code>break main</code>	在 main 函数入口设断点
<code>continue</code>	继续运行到断点
<code>next / step</code>	单步执行（不进入/进入函数）
<code>print 变量名</code>	查看变量值
<code>quit</code>	退出 GDB

说明：调试是人的工作，AI 不参与。这里只记录基本用法备查。如果需要图形化调试体验，可以在 VSCode 中安装 **Cortex-Debug** 扩展，它封装了上述 GDB + OpenOCD 流程。

4.4 STM32CubeProgrammer CLI（备用烧录工具，可选）

日常开发用 OpenOCD 就够了。CubeProgrammer 的优势在于 ST 专属操作（读写芯片保护位、OTP 区域、Option Bytes 等），遇到这些需求时再装不迟。

如果需要安装：

- 1. 访问 [STM32CubeProgrammer 下载页](#)（需要 myST 账号）
- 2. 下载 Linux 版（.zip），解压后运行安装脚本
- 3. 将 CLI 工具加入 PATH

安装完成后，烧录命令：

```
STM32_Programmer_CLI -c port=SWD -w build/firmware.bin 0x08000000 -v -rst
```

参数	说明
<code>-c port=SWD</code>	通过 SWD 协议连接
<code>-w ... 0x08000000</code>	写入固件到 Flash 起始地址
<code>-v</code>	校验
<code>-rst</code>	烧录后复位

五、验证清单

全部安装完成后，逐项验证。🔗 以下命令**逐条复制粘贴**执行：

```
# 1. ARM GCC 交叉编译器
arm-none-eabi-gcc --version
```

```
# 2. 二进制工具 (objcopy、size 等)
arm-none-eabi-objcopy --version
```

```
# 3. CMake
cmake --version
```

```
# 4. Ninja
ninja --version
```

```
# 5. GDB 多架构调试器
gdb-multiarch --version
```

```
# 6. OpenOCD
openocd --version
```

```
# 7. 串口工具
minicom --version && picocom --help 2>&1 | head -1
```

```
# 8. OpenOCD 配置文件是否存在 (确认有 ST-Link 和 STM32 配置模板)
ls /usr/share/openocd/scripts/interface/stlink.cfg
/usr/share/openocd/scripts/target/stm32f1x.cfg
```

Windows 侧验证 (在 PowerShell 中执行) :

```
# 9. usbipd-win
usbipd --version
```

10. STM32CubeMX – 从开始菜单打开, 确认能正常启动

全部通过即表示 STM32 嵌入式开发环境搭建完成。

六、常见问题

Q: `cmake -B build -G Ninja` 报错找不到编译器?

检查 CubeMX 生成的 `CMakeLists.txt` 或工具链文件中引用的编译器路径。如果写了绝对路径 (比如 CubeCLT 的路径), 需要改成 `arm-none-eabi-gcc` (不带路径, 让系统从 PATH 中查找)。

📄 确认编译器在 PATH 中:

```
which arm-none-eabi-gcc
```

Q: 编译报错 `arm-none-eabi-gcc: error: unrecognized command-line option?`

可能是 apt 版本的 GCC 不支持某些较新的编译选项。解决方法: 从 [ARM 官方](#) 下载最新版工具链替换。

Q: OpenOCD 报 `Error: open failed` 或 `libusb` 错误?

USB 设备权限问题。两种解法:

方法 1 (推荐): 添加 udev 规则, 📄 执行:

```
sudo tee /etc/udev/rules.d/99-stlink.rules > /dev/null << 'EOF'
# ST-Link V2
ATTRS{idVendor}=="0483", ATTRS{idProduct}=="3748", MODE="0666"
# ST-Link V2-1
ATTRS{idVendor}=="0483", ATTRS{idProduct}=="374b", MODE="0666"
# ST-Link V3
ATTRS{idVendor}=="0483", ATTRS{idProduct}=="374e", MODE="0666"
ATTRS{idVendor}=="0483", ATTRS{idProduct}=="374f", MODE="0666"
EOF
sudo udevadm control --reload-rules
```

添加规则后需要重新 `usbipd attach` 让设备重新挂载。

方法 2: 直接用 `sudo` 运行 OpenOCD:

```
sudo openocd -f interface/stlink.cfg -f target/stm32f1x.cfg \
-c "program build/firmware.elf verify reset exit"
```

WSL2 注意: WSL2 的 udev 支持取决于内核版本和 systemd 配置, 部分环境下 `udevadm` 可能不生效。如果添加 udev 规则后仍然报权限错误, 直接用 `sudo openocd` 作为长期方案完全没问题, 不必执着于 udev。

Q: `usbipd list` 看不到 ST-Link?

- 确认 ST-Link 已插入电脑, 且 Windows 设备管理器中能看到

- 确认 `usbipd-win` 服务正在运行：在 PowerShell 中执行 `Get-Service usbipd | Select-Object Status`
- 尝试换一个 USB 口

Q: `usbipd attach --wsl` 后 WSL 里 `lsusb` 看不到设备？

- 确认 WSL 内核版本足够新：在 WSL 中执行 `uname -r`，版本应 $\geq 5.10.60.1$
- 尝试 `wsl --shutdown` 后重新进入 WSL，再重新 `usbipd attach`

Q: CubeMX 的 Toolchain 选项里没有 CMake？

CubeMX 版本太旧。CMake 导出需要 **6.14.0 或更高版本**。在 CubeMX 中点击 `Help` → `About` 查看版本，如果低于 6.14.0 请更新。

Q: CubeMX 重新生成代码后，我的修改会被覆盖吗？

不会——前提是你把代码写在 `/* USER CODE BEGIN */` 和 `/* USER CODE END */` 注释标记之间。CubeMX 重新生成时会保留这些区域的内容。**写在标记外面的代码会被覆盖。**

Q: 编译出的 `.elf` 文件太大，超出芯片 Flash 容量？

📄 查看各段大小：

```
arm-none-eabi-size build/firmware.elf
```

常见原因：

- HAL 库默认开了很多不必要的外设驱动。在 CubeMX 中只勾选实际使用的外设，减少生成的代码量
- `printf` 等标准库函数会拉入大量代码。嵌入式场景可以用轻量替代（如 `nano specs`）
- 编译优化等级调高：在 `CMakeLists.txt` 中将 `-O0`（无优化）改为 `-Os`（优化体积）或 `-O2`（优化速度）

Q: 想在 VSCode 里获得代码补全和跳转？

安装以下 VSCode 扩展：

扩展	用途
CMake Tools	CMake 项目集成（配置、编译、选择构建目标）
clangd 或 C/C++	C 代码智能补全、跳转、语法检查
Cortex-Debug（可选）	图形化调试界面（封装 GDB + OpenOCD）

CMake Tools 会自动检测 `CMakeLists.txt` 并生成 `compile_commands.json`，clangd 读取这个文件就能提供准确的代码补全和跳转（包括 HAL 库函数）。

Q: 从 Keil 项目迁移到 CMake 怎么做?

1. 在 CubeMX 中打开原项目的 `.ioc` 文件 (CubeMX 配置文件, 如果有的话)
2. 将 Toolchain 改为 CMake, 重新生成
3. 把你在 Keil 项目里写的业务代码复制到新项目的 `/* USER CODE */` 区域
4. 如果没有 `.ioc` 文件, 需要在 CubeMX 中根据原项目的芯片型号和外设配置重新建一个项目